

Design and Implementation of FIR Filter using MATLAB and Verilog

Aslin Garvasis
EE24BTECH11008

1 Objective

The objective of this experiment is to design a Finite Impulse Response (FIR) low pass filter using MATLAB and implement the same filter in Verilog using three different architectures. The results obtained from MATLAB and Verilog are compared to verify the correctness of the hardware implementation.

2 Theory

A Finite Impulse Response (FIR) filter is a digital filter whose impulse response settles to zero in finite time. The output of the FIR filter is given by

$$y[n] = \sum_{k=0}^{N-1} h[k]x[n-k]$$

where

- $x[n]$ = input signal
- $y[n]$ = output signal
- $h[k]$ = filter coefficients
- N = number of taps

In this experiment the filter specifications are

- Sampling frequency = 10 kHz
- Cutoff frequency = 1 kHz
- Number of taps = 100

Three test signals of frequencies 950 Hz, 1100 Hz and 2000 Hz are applied to observe the filtering behavior.

3 MATLAB Implementation

MATLAB is used to design the FIR filter and generate the input signals. The filter coefficients are obtained using the `fir1()` function with a Hamming window.

```
fs = 10000;
fc = 1000;
N = 100;

wn = fc/(fs/2);
b_float = fir1(N-1, wn, hamming(N));
b_float = b_float / sum(b_float);
```

The coefficients are then converted into fixed point representation Q(2,14) and stored in a text file.

```
b_fixed = round(b_float * 2^14);
```

Three sinusoidal signals are generated using MATLAB.

```
x = sin(2*pi*f*t);
```

The signals are converted to fixed point format and stored in text files so that they can be used as inputs for the Verilog simulation.

4 FIR Filter Implementations

In this experiment, the FIR filter was implemented using three different approaches in order to study different hardware architectures. All three implementations compute the same mathematical equation but use different structures.

4.1 Direct Form FIR

The Direct Form implementation follows the FIR equation directly:

$$y[n] = \sum_{k=0}^{N-1} h[k]x[n-k]$$

This architecture uses a delay line to store previous input samples, multipliers to multiply each sample with the corresponding coefficient, and an adder chain to accumulate the results.

Theory

Each new input sample shifts the delay line. The delayed samples are multiplied by the filter coefficients and summed together to generate the output.

Code Explanation

The delay line is implemented using a shift register.

```
for (k = N:-1:2)
    delay_line(k) = delay_line(k-1);
end
delay_line(1) = x_fixed(n);
```

This code shifts the stored samples and inserts the new sample at the beginning of the delay line.

The multiply-accumulate operation is implemented as:

```
acc = 0;
for k = 1:N
    acc = acc + delay_line(k) * b_fixed(k);
end
```

This loop performs the FIR equation by multiplying each delayed sample with the corresponding coefficient and accumulating the results.

4.2 Optimized FIR Implementation

For linear phase FIR filters, the filter coefficients are symmetric:

$$h[k] = h[N - 1 - k]$$

Using this property, the FIR equation can be rewritten as:

$$y[n] = h[0](x[n] + x[n - (N - 1)]) + h[1](x[n - 1] + x[n - (N - 2)]) + \dots$$

Theory

Because the coefficients are symmetric, pairs of input samples can be added first and then multiplied by the same coefficient. This reduces the number of multipliers by half.

This optimization is important in hardware implementations since multipliers consume significant hardware resources.

Code Explanation

The symmetric computation is implemented as:

```
for k = 1:N/2
    acc = acc + b_fixed(k) * (delay_line(k) + delay_line(N-k+1));
end
```

Here, symmetric samples are added first and then multiplied with the corresponding coefficient.

4.3 Genvar Implementation

The third method mimics a hardware implementation where each tap operates in parallel. In Verilog, this is implemented using generate blocks with the **genvar** construct.

Theory

In hardware, each FIR tap performs a multiplication and the results are accumulated. Using a generate block allows multiple identical hardware structures to be created automatically.

Code Explanation

The multiplication stage is implemented as:

```

for k = 1:N
    mac_results(k) = delay_line(k) * b_fixed(k);
end

```

Each tap multiplies the delayed input with its corresponding coefficient.
The results are then accumulated:

```

acc = 0;
for k = 1:N
    acc = acc + mac_results(k);
end

```

This simulates the behavior of multiple MAC units operating in parallel in hardware.

All three methods produce identical outputs, which verifies that the implementations are mathematically equivalent.

5 Testbench Design

The testbench reads the input signal samples from text files generated by MATLAB. Each sample is applied to the FIR filter at every clock cycle. The output of the filter is written to an output text file.

These output files are later read by MATLAB for comparison with MATLAB generated outputs.

6 Results

6.1 Verification of FIR Implementations

The first figure shows the outputs obtained from the three FIR implementations: Direct form, Optimized form and Genvar implementation.

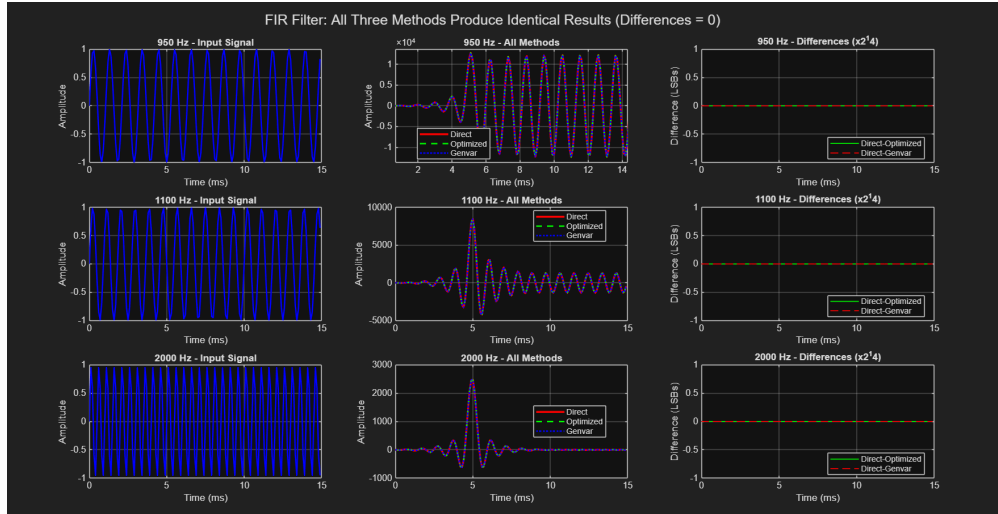


Figure 1: Comparison of Direct, Optimized and Genvar FIR Implementations

From the figure it can be observed that the outputs of all three implementations overlap perfectly. The difference plots are approximately zero, which confirms that all three FIR architectures produce identical results.

6.2 MATLAB vs Verilog Comparison

The second figure compares the MATLAB outputs with the Verilog simulation outputs for the three test signals.

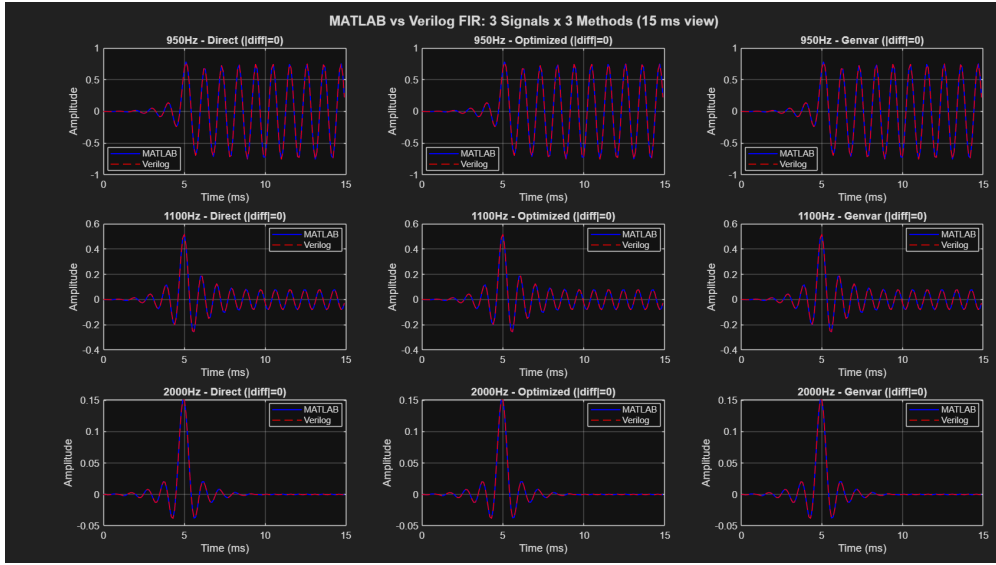


Figure 2: Comparison of MATLAB and Verilog FIR Outputs

The plots show that the Verilog outputs closely match the MATLAB outputs. This verifies that the hardware implementation of the FIR filter is correct.

For the 950 Hz signal, the output amplitude remains almost unchanged since it lies within the passband of the filter. The 1100 Hz signal lies near the cutoff frequency and shows partial attenuation. The 2000 Hz signal lies well beyond the cutoff frequency and is strongly attenuated.

7 Conclusion

In this experiment a 100 tap FIR low pass filter was designed using MATLAB and implemented in Verilog using three different architectures. The outputs obtained from the three hardware implementations were identical. The Verilog simulation results were also verified against MATLAB outputs and showed excellent agreement. This confirms the correctness of the FIR filter design and its hardware implementation.